

Hier ist die Analyse des PDF-Auszugs mit den Aufgaben und Lösungen:

Aufgabe 1: Fehlersuche

** (a) Fehler: Groß-/Kleinschreibung von `True` **

- **Fehler:** `true` ist falsch geschrieben, es muss `True` (mit großem `T`) sein.
- **Begründung:** In Python sind boolesche Werte `True` und `False` Großbuchstaben.

** (b) Fehler: Längeninkonsistenz zwischen Listen **

- **Fehler:** `stadt` hat 4 Elemente, `einwohner` nur 3 → `IndexError` bei `stadt[3]`.
- **Begründung:** `for i in range(len(stadt))` läuft bis 3, aber `einwohner` hat nur 3 Einträge.

** (c) Fehler: Keine Basis für Rekursion **

- **Fehler:** Keine Abbruchbedingung für die Rekursion (`n <= 0` fehlt).
- **Begründung:** Unendliche Rekursion führt zu `RecursionError`.

** (d) Fehler: Uninitialisierte Variable `i` **

- **Fehler:** `i` ist nicht definiert, bevor es in `while i <= 10` verwendet wird.
- **Begründung:** Python erkennt uninitialisierte Variablen nicht automatisch.

** (e) Fehler: Division durch 0 **

- **Fehler:** `int(b)` ergibt `0` → Division durch Null.
- **Begründung:** `b = 0.6` wird zu `0` bei `int(b)`.

Aufgabe 2: Ausgabe

** (a) Ausgabe: `False` **

- **Begründung:** `4 \* 4 = 16` und `2 \* 2 = 4` → `16 == 4` ist `False`.

** (b) Ausgabe: `3` **

- **Begründung:** `range(3, 4)` → nur `3` wird ausgegeben.

** (c) Ausgabe: `i`, `j`, `k` **

- **Begründung:** Schleife durchläuft `l = ["i", "j", "k"]` und druckt jedes Element.

** (d) Ausgabe: `t` **

- **Begründung:** `a[16]` bis `a[19]` → "macht", `a[5]` → "a", `a[9]` → "l", `a[2]` → "k", `a[18]` → "t", `a[1]` → "a".

** (e) Ausgabe: `1` bis `5` **

- **Begründung:** Rekursion von `k(1)` bis `k(5)` druckt `i` (1 bis 5).

Aufgabe 3: Run Length Encoding (RLE)

** (a) `encode(text)` **

- **Lösung:**

```
```python
def encode(text):
 result = []
 count = 1
```

```

for i in range(1, len(text)):
 if text[i] == text[i-1] and count < 9:
 count += 1
 else:
 result.append(f"{count}{text[i-1]}")
 count = 1
result.append(f"{count}{text[-1]}")
return "".join(result)
...

```

#### **(b) `decode(text)`**

- **Lösung:**

```

```python
def decode(text):
    result = []
    i = 0
    while i < len(text):
        if text[i].isdigit():
            count = int(text[i])
            result.append(text[i+1] * count)
            i += 2
        else:
            result.append(text[i])
            i += 1
    return "".join(result)
...

```

(c) RLE ineffizient bei:

- **Begründung:** Wenn der Text viele verschiedene Zeichen hat (z. B. "abcdefg"), wird die Länge erhöht.

(d) `checkRLE(text)`

- **Lösung:**

```

```python
def checkRLE(text):
 encoded = encode(text)
 return len(encoded) < len(text)
...

```

---

#### **Aufgabe 4: Schimpfwortfilter**

#### **(a) `readSwearwordData(filename)`**

- **Lösung:**

```

```python
def readSwearwordData(filename):
    try:
        with open(filename, "r") as file:
            return [[line.split()[0], int(line.split()[1])] for line in file]
    except:
        return []

```

...

****(b) `checkText(message, swearwords)`**

- ****Lösung:****

````python`

```
def checkText(message, swearwords):
 total_severity = 0
 words = message.split()
 for i, word in enumerate(words):
 for [swear, severity] in swearwords:
 if word.lower() == swear.lower():
 total_severity += severity
 words[i] = "****"
 if total_severity > 7:
 return "Nachricht wurde gelöscht"
 return " ".join(words)
```

...

#### **\*\*(c) Hauptprogramm**

- **\*\*Lösung:\*\***

````python`

```
print(checkText("Du bist natürlich kein Idiot", readSwearwordData("swearwords.txt")))
```

...

****Aufgabe 5: Java-Klasse `Moebel`**

- ****Lösung:****

````java`

```
public class Moebel {
 private String name;
 private int preis;

 public Moebel(String name, int preis) {
 this.name = name;
 this.preis = preis;
 }

 @Override
 public String toString() {
 return name + " (" + preis + " €)";
 }
}
```

...

---

**\*\*Hinweis:\*\*** Die Lösungen sind vereinfacht und können je nach Anforderungen angepasst werden.